Heap Data Structure

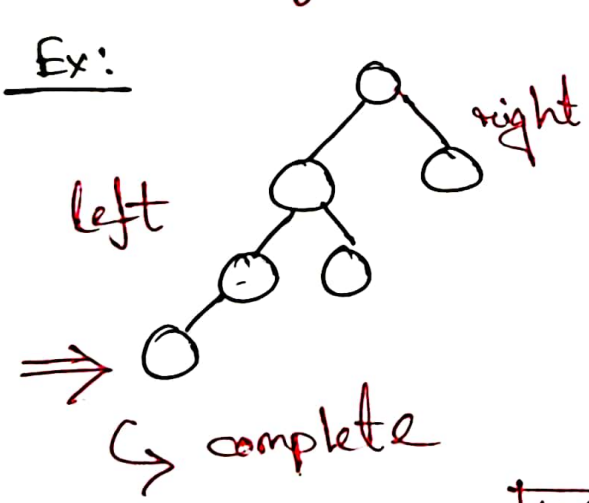
(*) Heap → Special Binary tree.
  └→ complete Binary tree.

(*) Complete Binary tree

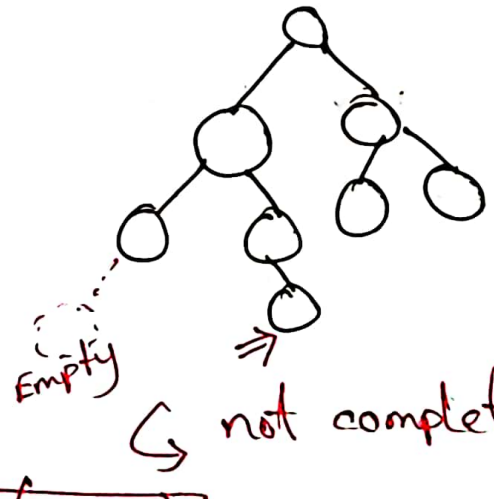
↳ last level node will start to fill up from the left.

(*) last level left side node can't be empty.

Ex:



↳ complete



↳ not complete.

special (reason)

(*) In heap structure, both children node will always have to be either bigger / smaller than parent node.

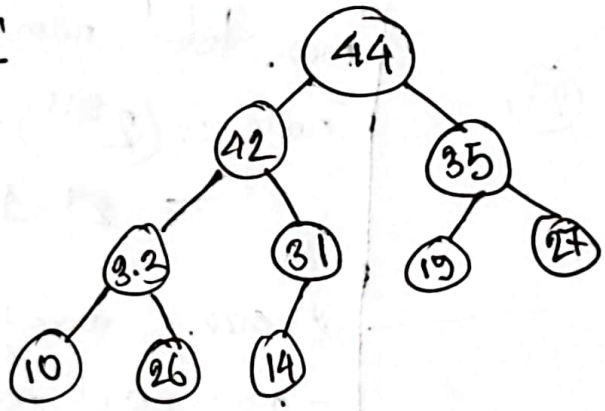
Heap → Max Data Structure (root will have the max value)
Heap → Min " " (root will have minimum value)

Max Heap

↳ parent node will always have bigger value.

(or) parent node >> child node

Ex:

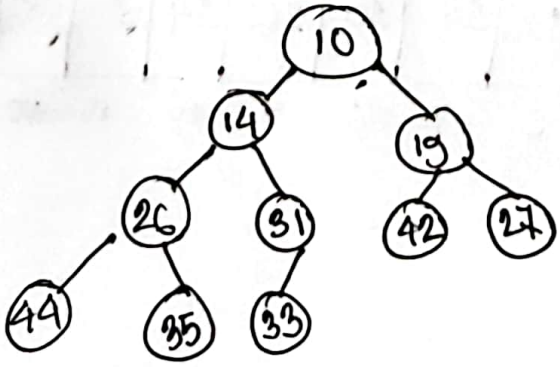


Min heap

↳ parent node will be smaller than child node

(or) parent << child node

Ex:



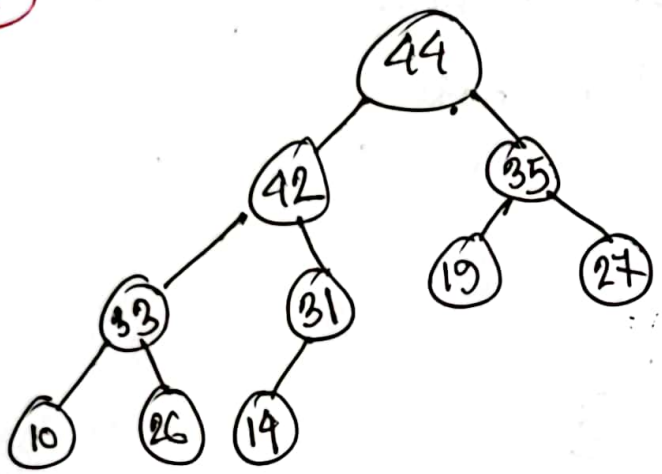
(or) No left/right rule mandatory. Just focus on parent node.

Healthcare
Emistat®
Ondansetron

Heap → ① Linked list based representation
↳ for simulation

② array based representation
↳ for coding.

array
↳ random access memory



Height = 3

max total num of node = $(2^{3+1}) - 1$
$= 2^4 - 1 = 15$

size of array
= max no of nodes + 1
= 15 + 1 = 16.
∴ we need a 16 size array.

root → index → 1

condition
root → 1st idx
parent → (i) idx
left child → (2i) idx
right child → (2i+1) idx

	44	42	35	33	31	19	27	10	26	14	None	None	None	None	None
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

l = 4, r = 5
l = 6, r = 7
l = 10, r = 11
l = 2 idx, r = 3 idx
l = 8, r = 9

Now

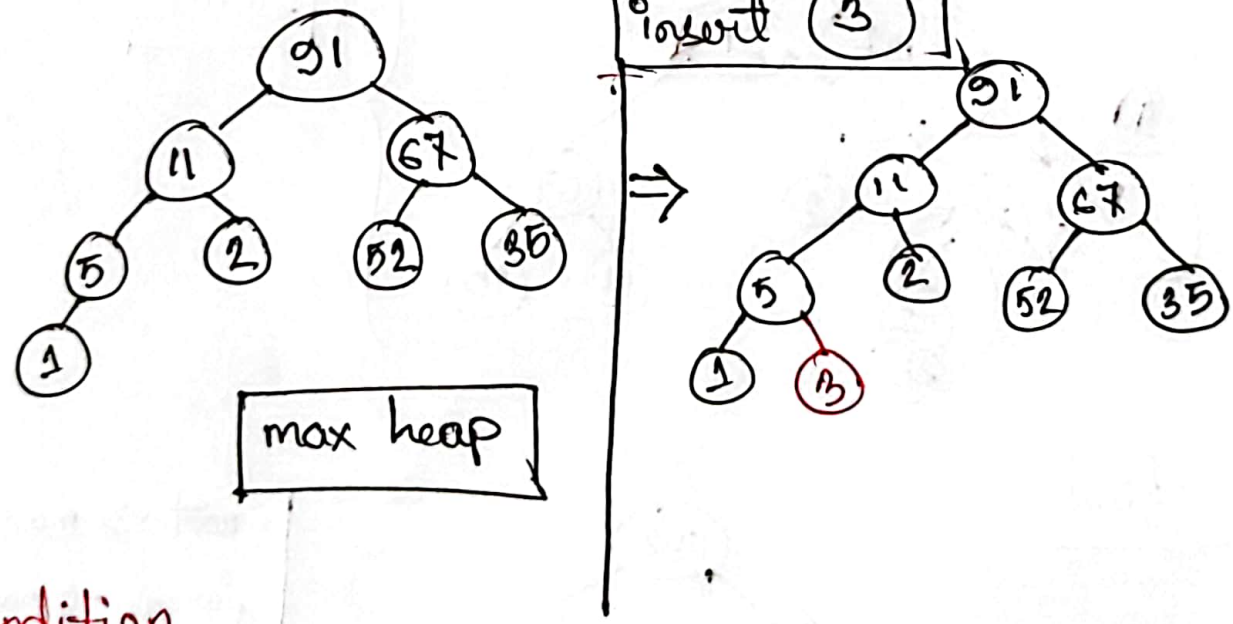
8th index Node. ⟶ 10

parent of 10 ⟶ 8//2 = 4th idx = 33 (value)

(#) ith idx node ⟶ x

∴ parent ⟶ i//2 th idx

Insertion



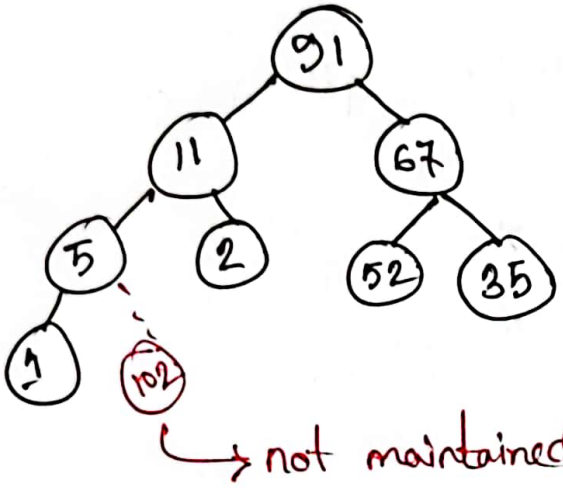
condition

(#) insert at the left most available node.

(#) maintain heap property. (check parent & child condition)

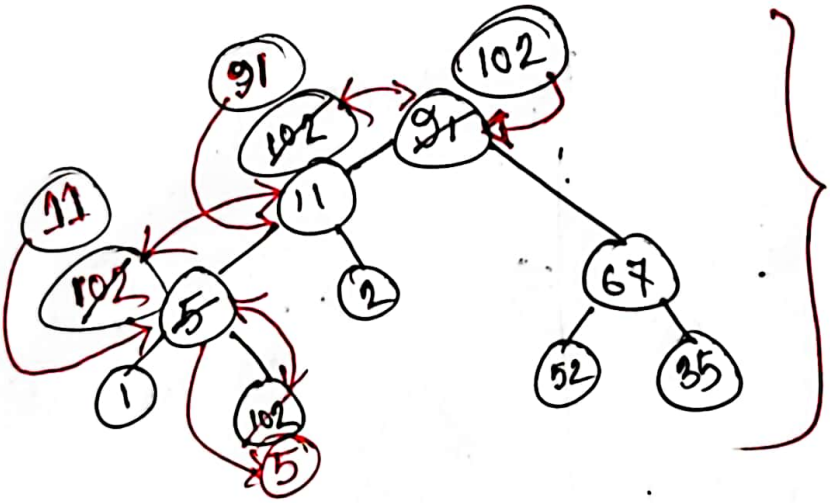
Healthcare
Emistat®
Ondansetron

insert 102



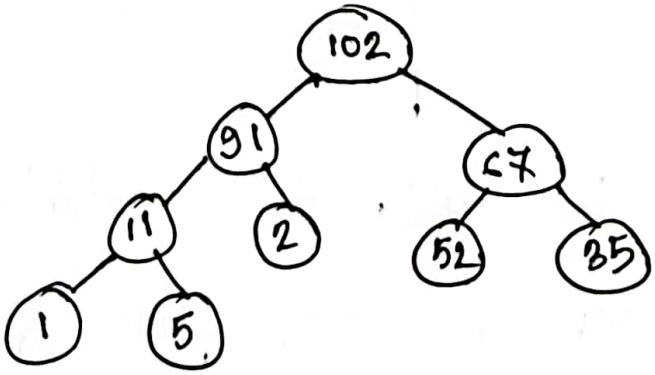
→ not maintained.

(1) insert leftmost avaiable
(2) as 102 > 5 → parent.
(3) swap replace parent node with the newly inserted node until heap property is maintained



swim

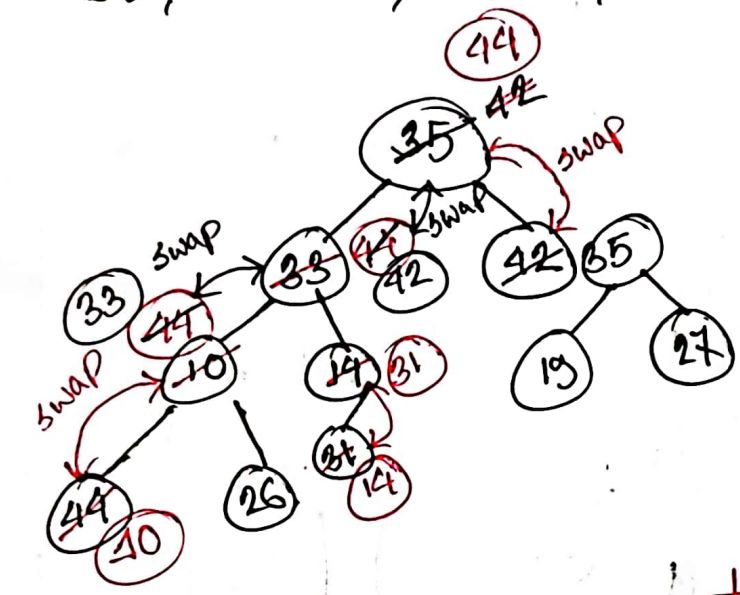
final



root → max
parent → maintained

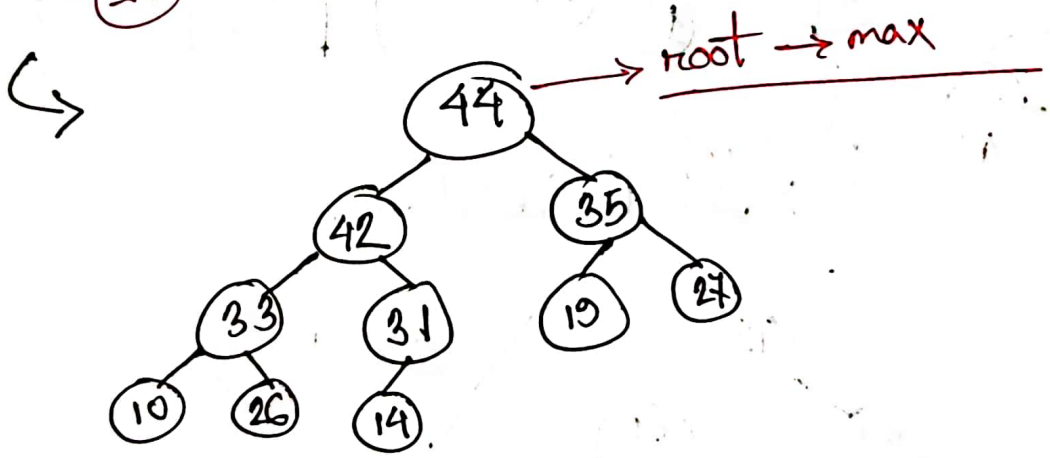
Insert values in a heap

35, 33, 42, 10, 14, 19, 27, 44, 26, 31



going upward

swim



Healthcare

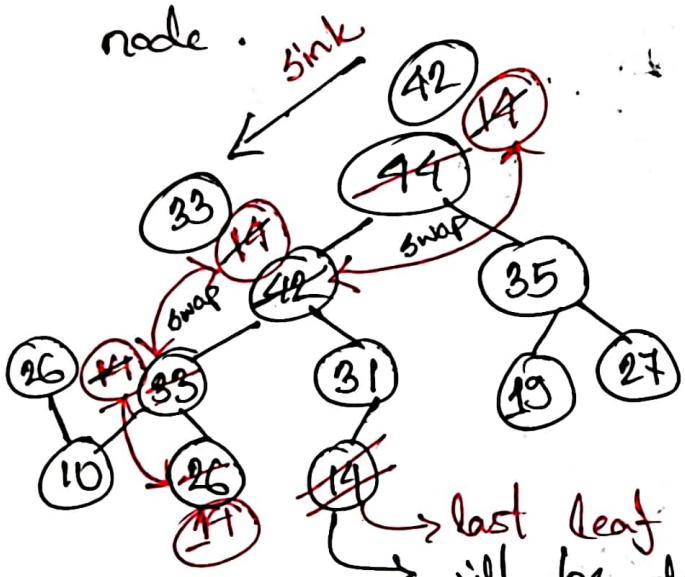
Emistat®
Ondansetron

Delete

↳ delete always from root.

Step

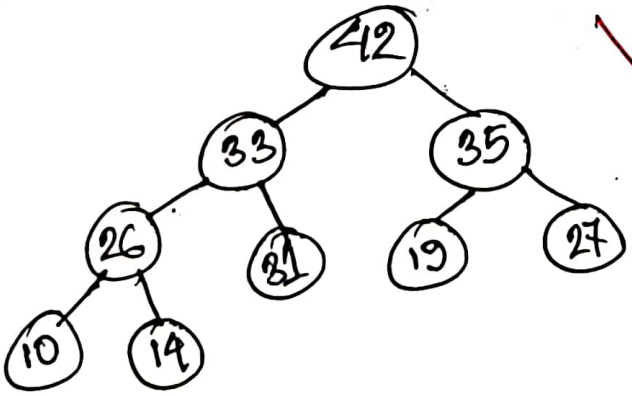
↳ replace the last leaf node with root node.



as heap property not maintained,

swap with larger value / child

→ last leaf node will be deleted.



sink method applied

Here, 44 deleted.

Heap Sort

35, 33, 42, 10, 14, 19, 27, 44, 26, 31

(or) delete consecutively from max heap

↳ consecutively larger numbers are deleted.

result

44, 42, 35, 33, 31, ...

↳ sorted in a descending order.

(or) followed by sink method & heap deletion.

(or) if inserted value is greater than the root, then swim method will be applied.

insertion

worst case → swim upto height amount times.

Healthcare

Emistat®
Ondansetron